



The Basics of Java Concurrency

1. Life Cycle of a Thread in Java

1. Introduction.....	2
2. Multithreading in Java.....	3
3. Life Cycle of a Thread in Java.....	4
3.1. New.....	4
3.2. Runnable.....	5
3.3. Blocked.....	6
3.4. Waiting.....	7
3.5. Timed Waiting.....	9
3.6. Terminated.....	10
4. Conclusion.....	12

2. How to Start a Thread in Java

1. Introduction.....	14
2. The Basics of Running a Thread.....	15
3. The <i>ExecutorService</i> Framework.....	17
4. Starting a Task With Executors.....	18
5. Starting a Task With <i>CompletableFutures</i>	19
6. Running Delayed or Periodic Tasks.....	20
6.1. <i>Timer</i>	20

6.2. <i>ScheduledThreadPoolExecutor</i>	21
6.3. Which Tool is Better?.....	22
7. Difference Between <i>Future</i> and <i>ScheduledFuture</i>	23
8. Conclusions.....	24
3. wait and notify() Methods in Java	
1. Overview.....	26
2. Thread Synchronization in Java.....	27
2.1. Guarded Blocks in Java.....	27
3. The <i>wait()</i> Method.....	28
3.1. <i>wait()</i>	28
3.2. <i>(long timeout)</i>	28
3.3. <i>(long timeout, int nanos)</i>	29
4. <i>notify()</i> and <i>notifyAll()</i>	30
4.1. <i>notify()</i>	30
4.2. <i>notifyAll()</i>	30
5. Sender-Receiver Synchronization Problem.....	31
5.1. Why Enclose <i>wait()</i> in a while Loop?.....	33
5.2. Why Do We Need to Synchronize <i>send()</i> and <i>receive()</i> Methods?.....	33
6. Conclusion.....	36



1. Life Cycle of a Thread in Java



In this chapter, we'll discuss a core concept in Java in detail: the lifecycle of a thread.

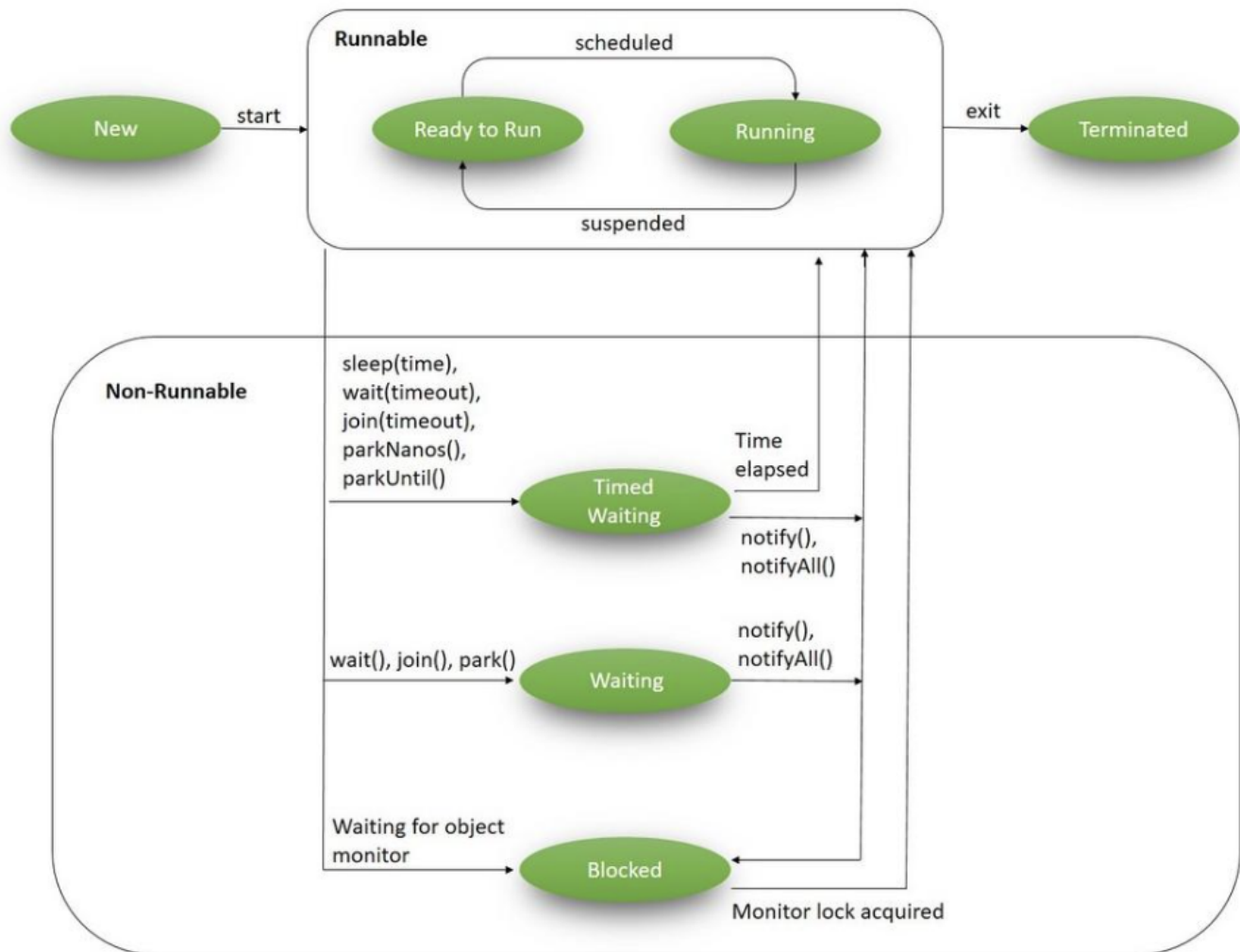
We'll use a quick illustrated diagram and practical code snippets to understand these states during the thread execution better.

To better understand Threads in Java, this article on creating a thread is a good place to start.

2. Multithreading in Java



In the Java language, multithreading is driven by the core concept of a Thread. During their lifecycle, threads go through various states:



3. Life Cycle of a Thread in Java



The `java.lang.Thread` class contains a static `State` enum that defines its potential states. During any given point in time, the thread can only be in one of these states:

NEW – a newly created thread that hasn't yet started the execution

RUNNABLE – either running or ready for execution but waiting for resource allocation

BLOCKED – waiting to acquire a monitor lock to enter or re-enter a synchronized block/method

WAITING – waiting for some other thread to perform a particular action without any time limit

TIMED_WAITING – waiting for some other thread to perform a specific action for a specified period

TERMINATED – has completed its execution

All of these states are covered in the diagram above; let's discuss each in detail.

3.1. New

A *NEW Thread* (or a *Born Thread*) is a thread that's been created but not yet started. It remains in this state until we start it using the `start()` method. The following code snippet shows a newly created thread that's in the **NEW** state:

```
1. Runnable runnable = new NewState();
2. Thread t = new Thread(runnable);
3. System.out.println(t.getState());
```

Since we've not yet started the mentioned thread, the method `t.getState()` prints:

NEW

3.2. Runnable

Once we've created a new thread and called the *start()* method on it, it's moved from the NEW to RUNNABLE state. **Threads in this state are either running or ready to run, but they're waiting for resource allocation from the system.**

In a multi-threaded environment, the Thread-Scheduler (part of JVM) allocates a fixed amount of time to each thread. So it runs for a particular time, then relinquishes the control to other RUNNABLE threads.

For example, let's add the *t.start()* method to our previous code and try to access its current state:

```
1. Runnable runnable = new NewState();
2. Thread t = new Thread(runnable);
3. t.start();
4. System.out.println(t.getState());
```

This code is most likely to return the output as:

RUNNABLE

Note that in this example, it's not always guaranteed that by the time our control reaches *t.getState()*, it will still be in the *RUNNABLE* state.

It may be that it was immediately scheduled by the Thread-Scheduler and has already finished execution. In such cases, we may get a different output.

3.3. Blocked

A thread is in the *BLOCKED* state when it's not currently eligible to run. **It enters this state while waiting for a monitor lock and trying to access a section of code that's locked by some other thread.**

Let's try to reproduce this state:

```
1. public class BlockedState {
2.     public static void main(String[] args) throws
3.     InterruptedException {
4.         Thread t1 = new Thread(new DemoBlockedRunnable());
5.         Thread t2 = new Thread(new DemoBlockedRunnable());
6.
7.         t1.start();
8.         t2.start();
9.
10.        Thread.sleep(1000);
11.
12.        System.out.println(t2.getState());
13.        System.exit(0);
14.    }
15. }
16.
17. class DemoBlockedRunnable implements Runnable {
18.     @Override
19.     public void run() {
20.         commonResource();
21.     }
22.
23.     public static synchronized void commonResource() {
24.         while(true) {
25.             // Infinite loop to mimic heavy processing
26.             // 't1' won't leave this method
27.             // when 't2' tries to enter this
28.         }
29.     }
30. }
```

In this code:

We've created two different threads: `t1` and `t2`.

`t1` starts and enters the synchronized `commonResource()` method, meaning only one thread can access it. All other subsequent threads that try to access this method will be blocked from further execution until the current one finishes processing.

When `t1` enters this method, it's kept in an infinite while loop; this is just to imitate heavy processing so that all other threads can't enter this method.

Now, when we start `t2`, it tries to enter the `commonResource()` method, which is already being accessed by `t1`; thus, `t2` will be kept in the `BLOCKED` state.

Being in this state, if we call `t2.getState()`, we'll get the output as:

`BLOCKED`

3.4. Waiting

A thread is in the `WAITING` state when it's waiting for some other thread to perform a particular action. [According to JavaDocs](#), any thread can enter this state by calling any one of the following three methods:

`object.wait()`
`thread.join()` or
`LockSupport.park()`

Note that in `wait()` and `join()`, we don't define any timeout period, as that scenario is covered in the next section.

We have a separate tutorial that discusses in detail the use of `wait()`, `notify()`, and `notifyAll()`.

For now, let's try to reproduce this state:

```

1. public class ingState implements Runnable {
2.     public static Thread t1;
3.
4.     public static void main(String[] args) {
5.         t1 = new Thread(new ingState());
6.         t1.start();
7.     }
8.
9.     public void run() {
10.        Thread t2 = new Thread(new DemoingStateRunnable());
11.        t2.start();
12.
13.        try {
14.            t2.join();
15.        } catch (InterruptedException e) {
16.            Thread.currentThread().interrupt();
17.            e.printStackTrace();
18.        }
19.    }
20. }
21.
22. class DemoingStateRunnable implements Runnable {
23.     public void run() {
24.         try {
25.             Thread.sleep(1000);
26.         } catch (InterruptedException e) {
27.             Thread.currentThread().interrupt();
28.             e.printStackTrace();
29.         }
30.
31.         System.out.println(ingState.t1.getState());
32.     }
33. }

```

Let's discuss what we're doing here:

We've created and started the *t1*.

t1 creates a *t2* and starts it.

While the processing of *t2* continues, we call *t2.join()*, which puts *t1* in the *WAITING* state until *t2* has finished execution.

Since *t1* is waiting for *t2* to complete, we're calling *t1.getState()* from *t2*.

As we expect, the output here is:

WAITING

3.5. Timed Waiting

A thread is in the *TIMED_WAITING* state when it s for another thread to perform a particular action within a stipulated time.

[According to JavaDocs](#), there are five ways to put a thread in the TIMED_WAITING state:

```
thread.sleep(long millis)
(int timeout) or (int timeout, int nanos)
thread.join(long millis)
LockSupport.parkNanos
LockSupport.parkUntil
```

To read more about the differences between *wait()* and *sleep()* in Java, look at [this dedicated article here](#).

For now, let's try to reproduce this state quickly:

```
1. public class TimedingState {
2.     public static void main(String[] args) throws InterruptedException {
3.         DemoTimeingRunnable runnable= new DemoTimeingRunnable();
4.         Thread t1 = new Thread(runnable);
5.         t1.start();
6.
7.         // The following sleep will give enough time for ThreadScheduler
8.         // to start processing of thread t1
9.         Thread.sleep(1000);
10.        System.out.println(t1.getState());
11.    }
12. }
13. class DemoTimeingRunnable implements Runnable {
14.     @Override
15.     public void run() {
16.         try {
17.             Thread.sleep(5000);
18.         } catch (InterruptedException e) {
19.             Thread.currentThread().interrupt();
20.             e.printStackTrace();
21.         }
22.     }
23. }
```

Here, we've created and started a thread *t1*, which is entered into the sleep state with a timeout period of 5 seconds. The output here will be:

TIMED_WAITING

3.6. Terminated

This is the state of a dead thread. **It's in the *TERMINATED* state when it has either finished execution or was terminated abnormally.**

We have a [dedicated article](#) that discusses different ways of stopping the thread.

Let's try to achieve this state in the following example:

```
1. public class TerminatedState implements Runnable {
2.     public static void main(String[] args) throws
3.     InterruptedException {
4.         Thread t1 = new Thread(new TerminatedState());
5.         t1.start();
6.         // The following sleep method will give enough time for
7.         // thread t1 to complete
8.         Thread.sleep(1000);
9.         System.out.println(t1.getState());
10.    }
11.
12.    @Override
13.    public void run() {
14.        // No processing in this block
15.    }
16. }
```

Here, while we've started thread *t1*, the very next statement, *Thread.sleep(1000)*, gives enough time for *t1* to complete, so this program gives us the output:

TERMINATED

In addition to the thread state, we can check the *isAlive()* method to determine whether the thread is alive. For instance, if we call the *isAlive()* method on this thread, we'll get:

```
Assert.assertFalse(t1.isAlive());
```

It returns *false*. Simply put, a **thread is alive if, and only if, it's been started and hasn't yet died.**

4. Conclusion



In this chapter, we learned about the life cycle of a thread in Java. We looked at all six states defined by the *Thread.State enum*, and reproduced them with quick examples.

Although the code snippets will give the same output in almost every machine, in some exceptional cases, we may get different outputs, as the exact behavior of Thread Scheduler can't be determined.

As always, the code snippets used here are available on [GitHub](#).



2. How to Start a Thread in Java



In this chapter, we'll explore different ways to start a thread and execute parallel tasks.

This is very useful, particularly when dealing with long or recurring operations that can't run on the main thread or where the UI interaction can't be put on hold while waiting for the operation's results.

To learn more about the details of threads, read our chapter on the Life Cycle of a Thread in Java.

2. The Basics of Running a Thread



We can easily write some logic that runs in a parallel thread by using the Thread framework.

Let's try a basic example by extending the *Thread* class:

```
1. public class NewThread extends Thread {
2.     public void run() {
3.         long startTime = System.currentTimeMillis();
4.         int i = 0;
5.         while (true) {
6.             System.out.println(this.getName() + ": New Thread is
7. running..." + i++);
8.             try {
9.                 // for one sec so it doesn't print too fast
10.                Thread.sleep(1000);
11.            } catch (InterruptedException e) {
12.                e.printStackTrace();
13.            }
14.            ...
15.        }
16.    }
17. }
```

Next, we can write a second class to initialize and start our thread:

```
1. public class SingleThreadExample {
2.     public static void main(String[] args) {
3.         NewThread t = new NewThread();
4.         t.start();
5.     }
6. }
```

We should call the *start()* method on threads in the NEW state (the equivalent of not started). Otherwise, Java will throw an instance of *IllegalThreadStateException* exception.

Now, let's assume we need to start multiple threads:

```
1. public class MultipleThreadsExample {  
2.     public static void main(String[] args) {  
3.         NewThread t1 = new NewThread();  
4.         t1.setName("MyThread-1");  
5.         NewThread t2 = new NewThread();  
6.         t2.setName("MyThread-2");  
7.         t1.start();  
8.         t2.start();  
9.     }  
10. }
```

Our code still looks quite simple and very similar to the examples we can find online.

Of course, **this is far from production-ready code, where it's critical to manage resources correctly to avoid too much context switching or too much memory usage.**

To get production-ready, we now need to write additional boilerplate to deal with:

- the consistent creation of new threads
- the number of concurrent live threads
- the threads deallocation - very important for daemon threads to avoid leaks

We can write our code for all of these case scenarios if we want to, but why should we reinvent the wheel.



The `ExecutorService` implements the Thread Pool design pattern (also called a replicated worker or worker-crew model) and takes care of the thread management we mentioned above. Plus, it adds some very useful features, like thread reusability and task queues.

Thread reusability, in particular, is very important. In a large-scale application, allocating and deallocating many thread objects creates a significant memory management overhead.

With worker threads, we minimize the overhead caused by thread creation.

To ease the pool configuration, `ExecutorService` comes with an easy constructor and some customization options, such as the type of queue, the minimum and maximum number of threads, and their naming convention.

For more details about the *ExecutorService*, please read our [Guide to the Java ExecutorService](#).

4. Starting a Task With Executors



Thanks to this powerful framework, we can switch our mindset from starting threads to submitting tasks.

Let's look at how we can submit an asynchronous task to our executor:

```
1. ExecutorService executor = Executors.newFixedThreadPool(10);  
2. ...  
3. executor.submit(() -> {  
4.     new Task();  
5. });
```

There are two methods we can use: *execute*, which returns nothing, and *submit*, which returns a *Future* encapsulating the computation's result.

For more information about Futures, please read our [Guide to java.util.concurrent.Future](#).

5. Starting a Task With *CompletableFutures*



To retrieve the final result from a Future object, we can use the get method available in the object, but this would block the parent thread until the end of the computation.

Alternatively, we could avoid the block by adding more logic to our task, but we'd have to increase the complexity of our code.

Java 1.8 introduced a new framework on top of the Future construct to better work with the computation's result: the *CompletableFuture*.

CompletableFuture implements *CompletableStage*, which adds a vast selection of methods to attach callbacks and avoid all the plumbing needed to run operations on the result after it's ready.

The implementation to submit a task is a lot simpler:

```
CompletableFuture.supplyAsync(() -> "Hello");
```

supplyAsync takes a *Supplier* containing the code we want to execute asynchronously, which is the lambda parameter in our case.

The task is now implicitly submitted to the *ForkJoinPool.commonPool()*, or we can specify the *Executor* we prefer as a second parameter.

To know more about *CompletableFuture*, please read our [Guide To *CompletableFuture*](#).



When working with complex web applications, we may need to run tasks at specific times, maybe even regularly.

Java has a few tools that can help us to run delayed or recurring operations:

- *java.util.Timer*
- *java.util.concurrent.ScheduledThreadPoolExecutor*

6.1. Timer

The *timer* is a facility to schedule tasks for future execution in a background thread.

Tasks may be scheduled for one-time execution or repeated execution at regular intervals.

Let's see what the code looks like if we want to run a task after one second of delay:

```
1. TimerTask task = new TimerTask() {
2.     public void run() {
3.         System.out.println("Task performed on: " + new Date() +
4.         "n"
5.         + "Thread's name: " + Thread.currentThread().getName());
6.     }
7. };
8. Timer timer = new Timer("Timer");
9. long delay = 1000L;
10. timer.schedule(task, delay);
```

Now, let's add a recurring schedule:

```
timer.scheduleAtFixedRate(repeatedTask, delay, period);
```

This time, the task will run after the delay specified, and it'll be recurrent after the period of time has passed.

For more information, please read our guide to [Java Timer](#).

6.2. *ScheduledThreadPoolExecutor*

ScheduledThreadPoolExecutor has methods similar to the *Timer* class:

```
1. ScheduledExecutorService executorService = Executors.  
2.   newScheduledThreadPool(2);  
3. ScheduledFuture<Object> resultFuture  
4.   = executorService.schedule(callableTask, 1, TimeUnit.SECONDS);
```

To end our example, we'll use *scheduleAtFixedRate()* for recurring tasks:

```
1. ScheduledFuture<Object> resultFuture  
2.   = executorService.scheduleAtFixedRate(runnableTask, 100, 450,  
3.   TimeUnit.MILLISECONDS);
```

The code above will execute a task after an initial delay of 100 milliseconds, and after that, it'll execute the same task every 450 milliseconds.

If the processor can't finish processing the task in time before the next occurrence, the *ScheduledExecutorService* will wait until the current task is completed before starting the next.

To avoid this waiting time, we can use *scheduleWithFixedDelay()*, which, as described by its name, guarantees a fixed length delay between iterations of the task.

For more details about *ScheduledExecutorService*, please read our Guide to the Java ExecutorService.

6.3. Which Tool is Better?

If we run the examples above, the computation's result looks the same.

So, how do we choose the right tool?

When a framework offers multiple choices, it's important to understand the underlying technology to make an informed decision.

Let's try to dive a bit deeper under the hood.

Timer:

- doesn't offer real-time guarantees; it schedules tasks using the *Object.(long)* method
- there's a single background thread, so tasks run sequentially, and a long-running task can delay others
- runtime exceptions thrown in a *TimerTask* would kill the only thread available, thus killing *Timer*

ScheduledThreadPoolExecutor:

- can be configured with any number of threads
- can take advantage of all available CPU cores
- catches runtime exceptions and lets us handle them if we want to (by overriding the *afterExecute* method from *ThreadPoolExecutor*)
- cancels the task that threw the exception while letting others continue to run
- relies on the OS scheduling system to keep track of time zones, delays, solar time, etc.
- provides collaborative API if we need coordination between multiple tasks, like waiting for the completion of all tasks submitted
- provides better API for management of the thread life cycle

The choice is obvious now, right?



In our code examples, we can observe that *ScheduledThreadPoolExecutor* returns a specific type of *Future*: *ScheduledFuture*.

ScheduledFuture extends both *Future* and *Delayed* interfaces, thus inheriting the additional method *getDelay*, which returns the remaining delay associated with the current task. It's extended by *RunnableScheduledFuture*, which adds a method to check if the task is periodic.

ScheduledThreadPoolExecutor implements all of these constructs through the inner class *ScheduledFutureTask*, and uses them to control the task life cycle.



In this chapter, we experimented with the different frameworks available to start threads and run tasks in parallel.

Then, we explored in greater detail the differences between *Timer* and *ScheduledThreadPoolExecutor*.

The source code for the chapter is available [over on GitHub](#).



3. wait and notify() Methods in Java



In this chapter, we'll look at one of the most fundamental mechanisms in Java: thread synchronization.

First, we'll discuss some essential concurrency-related terms and methodologies.

Then, we'll develop a simple application where we'll deal with concurrency issues, with the goal of better understanding *wait()* and *notify()*.

2. Thread Synchronization in Java



In a multithreaded environment, multiple threads might try to modify the same resource. Not managing threads properly will lead to consistency issues.

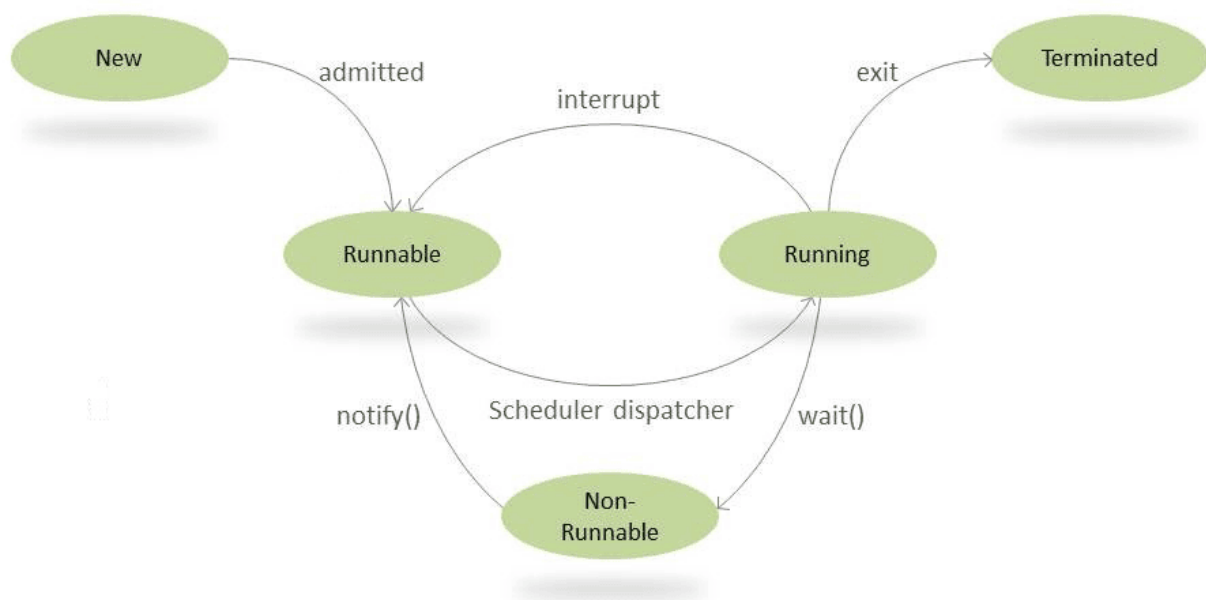
2.1. Guarded Blocks in Java

One tool we can use to coordinate the actions of multiple threads in Java is guarded blocks. Such blocks keep a check for a particular condition before resuming the execution.

With that in mind, we'll make use of the following:

- ***Object.()*** to suspend a thread
- ***Object.notify()*** to wake up a thread

We can better understand this through the following diagram depicting the life cycle of a *Thread*:



Please note that there are many ways of controlling this life cycle. However, in this article, we will focus only on *wait()* and *notify()*.

3. The *wait()* Method



Simply put, calling *wait()* forces the current thread to wait until some other thread invokes *notify()* or *notifyAll()* on the same object.

For this, the current thread must own the object's monitor. According to [Javadocs](#), this can happen in the following ways:

- when we've executed the *synchronized* instance method for the given object
- when we've executed the body of a *synchronized* block on the given object
- by executing *synchronized static* methods for objects of type *Class*

Note that only one active thread can own an object's monitor at a time.

This *wait()* method comes with three overloaded signatures. Let's have a look at these.

3.1. *wait()*

The *wait()* method causes the current thread to wait indefinitely until another thread either invokes *notify()* for this object or *notifyAll()*.

3.2. *wait(long timeout)*

We can specify a timeout using this method, after which a thread will be woken up automatically. A thread can be woken up before reaching the timeout using *notify()* or *notifyAll()*.

Note that calling *(0)* is the same as calling *wait()*.

3.3. *wait(long timeout, int nanos)*

This is yet another signature providing the same functionality. The only difference here is that we can provide higher precision.

The total timeout period (in nanoseconds) is calculated as $1_000_000 * \text{timeout} + \text{nanos}$.



We use the *notify()* method to wake up threads waiting for access to this object's monitor.

There are two ways of notifying waiting threads.

4.1. *notify()*

For all threads waiting on this object's monitor (by using any one of the *wait()* methods), the method *notify()* notifies any one of them to wake up arbitrarily. The choice of which thread to wake is nondeterministic and depends on implementation.

Since *notify()* wakes up a single random thread, we can use it to implement mutually exclusive locking where threads are doing similar tasks. But in most cases, it would be more viable to implement *notifyAll()*.

4.2. *notifyAll()*

This method simply wakes all threads waiting on this object's monitor.

The awakened threads will compete in the usual manner, like any other thread trying to synchronize on this object.

But before we allow their execution to continue, we should always **define a quick check for the condition required to proceed with the thread**. This is because there may be some situations where the thread woken up without receiving a notification (this scenario is discussed later in an example).

5. Sender-Receiver Synchronization Problem



Now that we understand the basics let's go through a simple *Sender-Receiver* application that will make use of the *wait()* and *notify()* methods to set up synchronization between them:

- The *Sender* is supposed to send a data packet to the Receiver.
- The *Receiver* can't process the data packet until the *Sender* finishes sending it.
- Similarly, the *Sender* shouldn't attempt to send another packet unless the *Receiver* has already processed the previous packet.

First, Let's create a Data class consisting of the data packet sent from the *Sender* to the *Receiver*. We'll use *wait()* and *notifyAll()* to set up synchronization between them:

```

1. public class Data {
2.     private String packet;
3.
4.     // True if receiver should
5.     // False if sender should
6.     private boolean transfer = true;
7.
8.     public synchronized String receive() {
9.         while (transfer) {
10.            try {
11.                ();
12.            } catch (InterruptedException e) {
13.                Thread.currentThread().interrupt();
14.                System.err.println("Thread Interrupted");
15.            }
16.        }
17.        transfer = true;
18.
19.        String returnPacket = packet;
20.        notifyAll();
21.        return returnPacket;
21.    }
22.
23.    public synchronized void send(String packet) {
24.        while (!transfer) {
25.            try {
26.                ();
27.            } catch (InterruptedException e) {
28.                Thread.currentThread().interrupt();
29.                System.err.println("Thread Interrupted");
30.            }
31.        }
32.        transfer = false;
33.
34.        this.packet = packet;
35.        notifyAll();
36.    }
37. }

```

Let's break down what's going on here:

- The `packet` variable denotes the data that's being transferred over the network.
- We have a boolean variable `transfer`, which the Sender and Receiver will use for synchronization:
- If this variable is `true`, the Receiver should for the Sender to send the message.
- If it's `false`, the Sender should for the Receiver to receive the message.
- The Sender uses the `send()` method to send data to the Receiver:
- If the `transfer` is `false`, we'll by calling `wait()` on this thread.
- But when it's `true`, we toggle the status, set our message, and call `notifyAll()` to wake up other threads to specify that a significant event has occurred, and they can check if they can continue execution.
- Similarly, the Receiver will use the `receive()` method:
- If the `transfer` was set to `false` by the Sender, only then will it proceed; otherwise, we'll call `wait()` on this thread.
- When the condition is met, we toggle the status, notify all waiting threads to wake up, and return the received data packet.

5.1. Why Enclose `wait()` in a while Loop?

Since `notify()` and `notifyAll()` randomly wake up threads waiting on this object's monitor, it's not always important to meet the condition. Sometimes, the thread is woken up, but the condition isn't actually satisfied yet.

We can also define a check to save us from spurious wakeups, where a thread can wake up from waiting without ever having received a notification.

5.2. Why Do We Need to Synchronize `send()` and `receive()` Methods?

We placed these methods inside `synchronized` methods to provide intrinsic locks. If a thread calling `wait()` method doesn't own the inherent lock, an error will be thrown.

We'll now create the *Sender* and *Receiver* and implement the *Runnable* interface on both so that their instances can be executed by a thread.

First, we'll see how *Sender* will work:

```
1. public class Sender implements Runnable {
2.     private Data data;
3.
4.     // standard constructors
5.
6.     public void run() {
7.         String packets[] = {
8.             "First packet",
9.             "Second packet",
10.            "Third packet",
11.            "Fourth packet",
12.            "End"
13.        };
14.
15.        for (String packet : packets) {
16.            data.send(packet);
17.
18.            // Thread.sleep() to mimic heavy server-side
19.            processing
20.            try {
21.                Thread.sleep(ThreadLocalRandom.current().
22.                    nextInt(1000, 5000));
23.            } catch (InterruptedException e) {
24.                Thread.currentThread().interrupt();
25.                System.err.println("Thread Interrupted");
26.            }
27.        }
28.    }
29. }
```

Let's take a closer look at this *Sender*:

- We're creating some random data packets that will be sent across the network in *packets[]* array.
- For each packet, we're merely calling *send()*.
- Then, we call *Thread.sleep()* with random intervals to mimic heavy server-side processing.

Finally, let's implement our *Receiver*:

```

1. public class Receiver implements Runnable {
2.     private Data load;
3.
4.     // standard constructors
5.
6.     public void run() {
7.         for(String receivedMessage = load.receive();
8.             !"End".equals(receivedMessage);
9.             receivedMessage = load.receive()) {
10.
11.             System.out.println(receivedMessage);
12.
13.             //Thread.sleep() to mimic heavy server-side processing
14.             try {
15.                 Thread.sleep(ThreadLocalRandom.current().nextInt(1000,
16. 5000));
17.             } catch (InterruptedException e) {
18.                 Thread.currentThread().interrupt();
19.                 System.err.println("Thread Interrupted");
20.             }
21.         }
22.     }
23. }

```

Here, we're simply calling *load.receive()* in the loop until we get the last "End" data packet. Let's now see this application in action:

```

1. public static void main(String[] args) {
2.     Data data = new Data();
3.     Thread sender = new Thread(new Sender(data));
4.     Thread receiver = new Thread(new Receiver(data));
5.
6.     sender.start();
7.     receiver.start();
8. }

```

We'll receive the following output:

```

First packet
Second packet
Third packet
Fourth packet

```

And here we are. **We've received all data packets in the right, sequential order** and successfully established the correct communication between our sender and receiver.



In this chapter, we discussed some core synchronization concepts in Java. More specifically, we focused on using *wait()* and *notify()* to solve interesting synchronization problems. Finally, we went through a code sample where we applied these concepts in practice.

Before we close, it's worth mentioning that all these low-level APIs, such as *wait()*, *notify()* and *notifyAll()*, are traditional methods that work well. Still, higher-level mechanisms are often simpler and better, such as Java's native *Lock* and *Condition* interfaces (available in *java.util.concurrent.locks* package).

For more information on the *java.util.concurrent* package, visit our [overview of the java.util.concurrent](#) article. Lock and Condition are covered in the [guide to java.util.concurrent.Locks](#).

The source code for the chapter is available [over on GitHub](#).